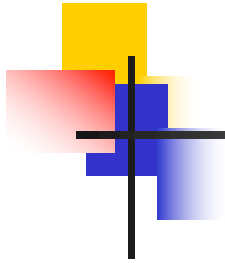
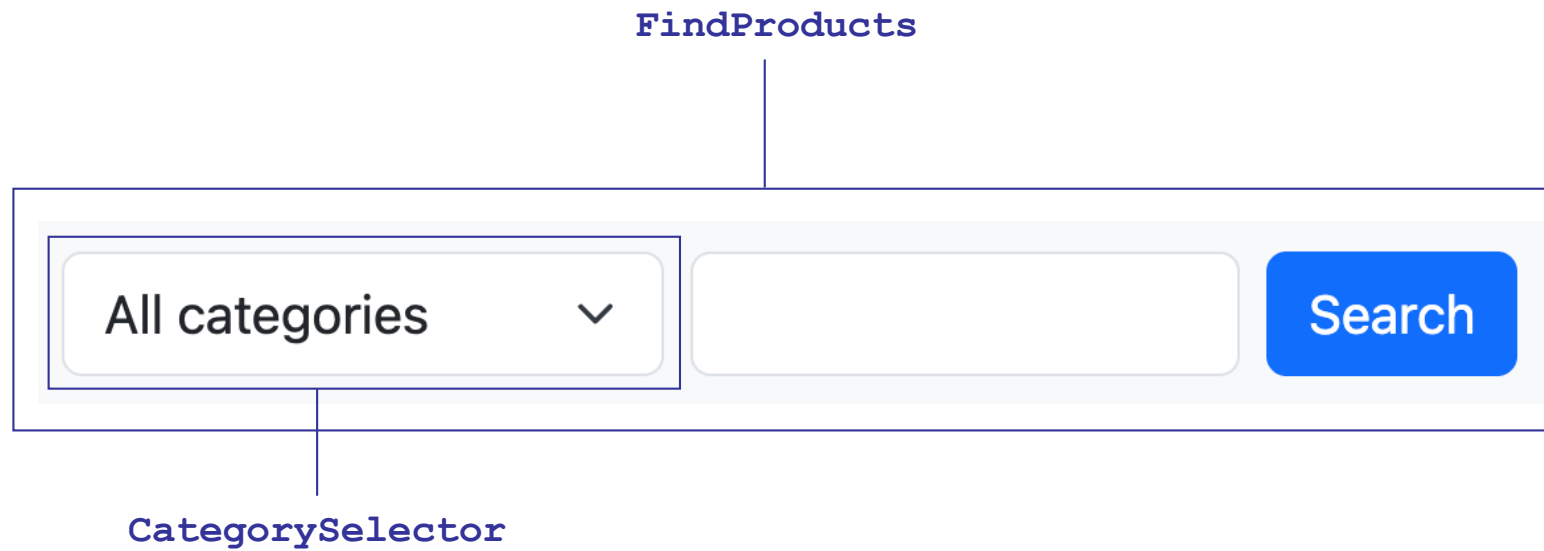


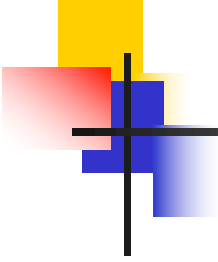
7.2 Caso de uso “buscar productos”

- Componentes
 - Uncontrolled components
 - Controlled components
 - Estado global
- Acciones
- Reductor



Componentes I





Componente FindProducts

...

```
import {useNavigate} from 'react-router';
```

```
import CategorySelector from '../CategorySelector';
```

```
import * as actions from '../actions';
```

```
import backend from '../../backend';
```

```
const FindProducts = () => {
```

```
    const dispatch = useDispatch();
```

```
    const navigate = useNavigate();
```

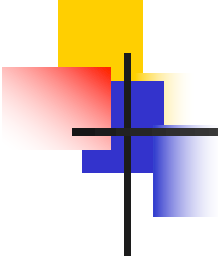
→ [React Router] Hook

```
    const [categoryId, setCategoryId] = useState('');
```

```
    const [keywords, setKeywords] = useState('');
```

```
    const toNumber = value => value.length > 0 ? Number(value) : null;
```

Estado local



Componente FindProducts

```
const handleSubmit = async event => {  
  
  event.preventDefault();  
  dispatch(actions.clearProductSearch());  
  navigate('/catalog/find-products-result');  
  
  const criteria = {categoryId: toNumber(categoryId),  
    keywords: keywords.trim(), page: 0};  
  const response = await backend.catalogService.findProducts(  
    criteria);  
  
  if (response.ok) {  
    dispatch(actions.findProductsCompleted(  
      {criteria, result: response.payload}));  
  }  
}
```

[React Router] Provoca un cambio de pantalla

Componente FindProducts

```
return (
```

```
  <Form onSubmit={e => handleSubmit(e)} className="d-flex">
```

Ver código fuente
(módulo catalog)

```
    <CategorySelector id="categoryId"  
      value={categoryId}  
      onChange={e => setCategoryId(e.target.value)} />
```

```
    <Form.Control id="keywords" type="text" className="me-2"  
      value={keywords}  
      onChange={e => setKeywords(e.target.value)} />
```

```
    <Button type="submit">  
      <FormattedMessage  
        id='project.global.buttons.search' />  
    </Button>
```

```
  </Form>
```

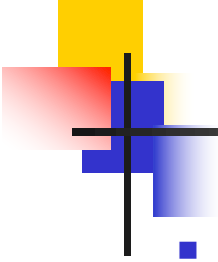
```
);
```

```
}
```

```
export default FindProducts;
```

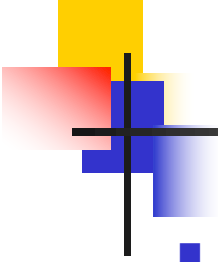
React Bootstrap

- Form genera el tag `form`, `Form.Control` genera el tag `input` y `Button` genera el tag `button`.
- Entre otros, reciben los mismos atributos que los tags HTML que generan y añaden estilos CSS adicionales



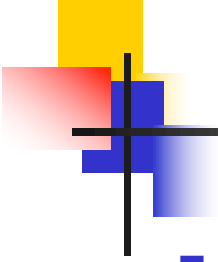
Uncontrolled Components

- Tanto en `pa-todos` como en `pa-todos-redux`, el componente `AddTodo` accedía al único campo de entrada del formulario mediante el uso del atributo especial `ref`
 - <https://github.com/udc-fic-pa/pa-todos-redux/blob/master/src/AddTodo.jsx>
- A los componentes que implementan formularios usando este estilo, se les llama **uncontrolled components**
 - Los datos del formulario no se guardan en el estado del componente ni en Redux, sino directamente en el DOM
- Cuando el componente desea conocer el valor de algún campo, lo consulta accediendo al nodo correspondiente del DOM
 - Estilo “pull”: el componte coge los datos del DOM
- Aunque es un estilo que requiere poco código, en general, no es la solución recomendada desde la documentación de React
 - Parte del estado de la IU está fuera del control de React (y/o Redux)
 - En algunos casos complicaría la implementación de funcionalidades adicionales



Controlled components

- **FindProducts** está implementado con otro estilo
 - Los valores de los campos del formulario se guardan en el **estado local** del componente
 - Sólo son de interés para el componente
 - Cada campo de entrada usa
 - **value** para establecer el valor del campo
 - **onChange** para actualizar el estado local cada vez que el usuario modifica el valor del campo
 - Cada vez que se modifica el valor de un campo, el estado local se actualiza
 - Estilo “push”: el componente recibe los datos
- A los componentes que implementan formularios usando este estilo, se les llama **controlled components**



Controlled components

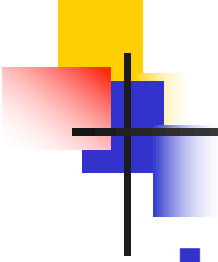
- Con respecto a un uncontrolled component, un controlled component
 - Requiere un poco más de código (gestión del estado local)
 - Pero hace más sencillo incorporar funcionalidades adicionales, como por ejemplo, modificar los datos de entrada del usuario o realizar validaciones mientras teclea
 - Ejemplo: forzar que los datos de un campo siempre aparezcan en mayúsculas mientras se teclea

```
const [name, setName] = useState('');
```

```
...
```

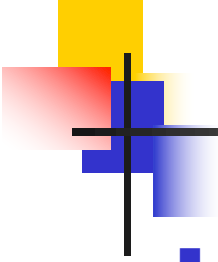
```
<Form.Control id="name" type="text" className="..."  
  value={name}  
  onChange={e => setName(e.target.value.toUpperCase())}/>
```

```
...
```



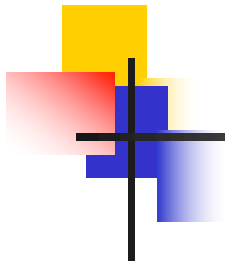
Estado global

- En el **estado global** (Redux) se guarda un objeto con información sobre la búsqueda
 - Se establece desde `FindProducts` y se recupera desde `FindProductsResult`
- El objeto con información de la búsqueda contiene dos campos
 - **criteria**: criterios de búsqueda
 - `categoryId`
 - `keywords`
 - `page`
 - **result**: resultado de la búsqueda
 - `items`
 - `existMoreItems`

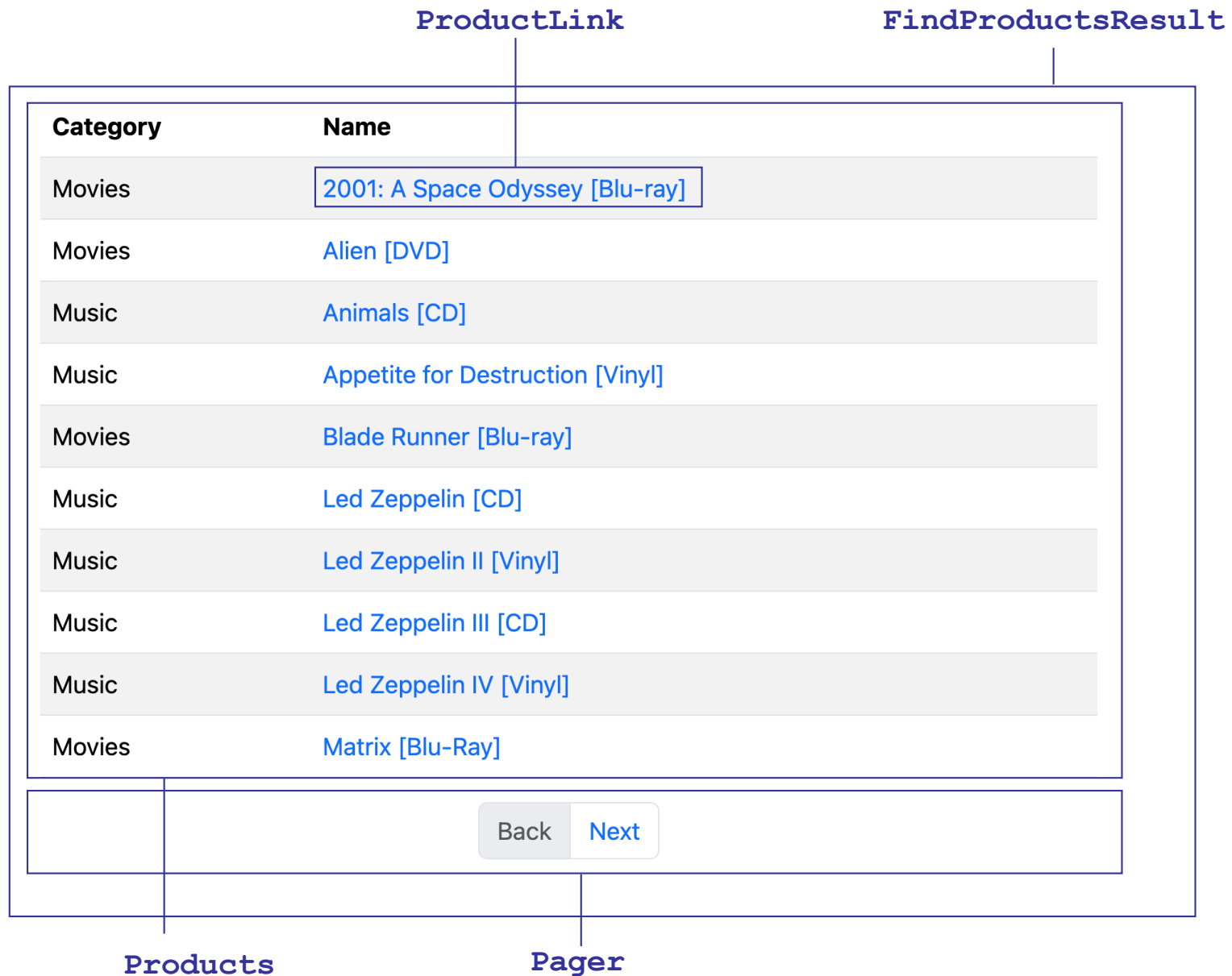


Flujo de la búsqueda

- Cuando el usuario hace clic en el botón de buscar (`handleSubmit`)
 - Se anula la información de la búsqueda anterior en el estado global
 - `actions.clearProductSearch`
 - Se pasa a la pantalla del resultado de la búsqueda
 - `navigate('/catalog/find-products-result')`
 - Inicialmente muestra una pantalla en blanco
 - Se hace una petición al backend para realizar la búsqueda
 - `backend.catalogService.findProducts`
 - Cuando llega la respuesta del backend
 - Se establece la información de la búsqueda en el estado global
 - `actions.findProductsCompleted`
 - La pantalla del resultado de la búsqueda (`FindProductsResult`), que está suscrita a cambios en el estado global usando un selector apropiado, se vuelve a renderizar mostrando el resultado de la búsqueda



Componentes II



Componente FindProductsResult

...

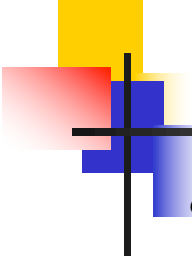
```
import * as selectors from '../selectors';
import * as actions from '../actions';
import backend from '../../backend';
import {Pager} from '../common';
import Products from './Products';
```

```
const FindProductsResult = () => {
```

Estado global

```
const productSearch = useSelector(selectors.getProductSearch);
const categories = useSelector(selectors.getCategories);
const dispatch = useDispatch();
```

productSearch contiene criteria
(categoryId, keywords y page) y result
(items y existMoreItems)



Componente FindProductsResult

```
const handleBackNext = async (backClicked, criteria) => {  
  
  dispatch(actions.clearProductSearch() );  
  
  let newCriteria;  
  
  if (backClicked) {  
    newCriteria = {...criteria, page: criteria.page-1}  
  } else {  
    newCriteria = {...criteria, page: criteria.page+1}  
  }  
  
  const response = await backend.catalogService.findProducts(  
    newCriteria);  
  
  if (response.ok) {  
    dispatch(actions.findProductsCompleted(  
      {criteria: newCriteria, result: response.payload}));  
  }  
}
```

Componente FindProductsResult

```
if (!productSearch) {  
  return null;  
}
```

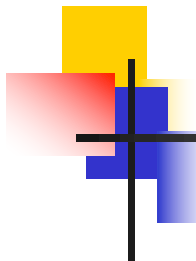
Si en el estado global `catalog.productSearch` todavía es `null`, el componente no renderiza nada

```
if (productSearch.result.items.length === 0) {  
  return (  
    <Alert variant="info">  
      <FormattedMessage id='...noProductsFound' />  
    </Alert>  
  );  
}
```

React Bootstrap

- `Alert` genera el tag `div` añadiendo estilos CSS

No products found



Componente FindProductsResult

```
return (
```

```
<>
```

```
  <Products products={productSearch.result.items}  
    categories={categories}/>
```

Ver código fuente
(módulo common)



```
  <Pager
```

```
    back={{  
      enabled: productSearch.criteria.page >= 1,  
      onClick: () => handleBackNext(true,  
        productSearch.criteria)}}  
    next={{  
      enabled: productSearch.result.existMoreItems,  
      onClick: () => handleBackNext(false,  
        productSearch.criteria)}}/>
```

```
</>
```

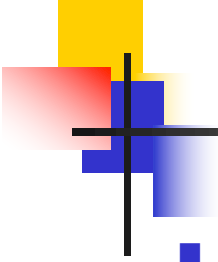
```
);
```

```
}
```

```
export default FindProductsResult;
```

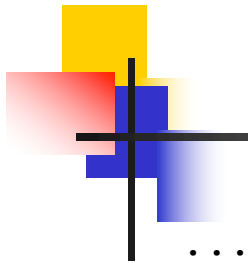
React

- `<>`: elemento especial de React que se puede usar cuando el markup devuelto no incluye un elemento raíz de forma natural
- No genera ningún elemento HTML



Flujo Back/Next

- Cuando el usuario hace clic en Back o Next (`handleBackNext`)
 - Se anula la información de la búsqueda anterior en el estado global
 - `actions.clearProductSearch`
 - La pantalla del resultado de la búsqueda, que está suscrita a cambios en el estado global usando un selector apropiado (`selectors.getProductSearch`), se vuelve a renderizar eliminando los resultados de la búsqueda anterior (`return null`)
 - Se hace una petición al backend para realizar la nueva búsqueda
 - `backend.catalogService.findProducts`
 - Cuando llega la respuesta del backend
 - Se establece la información de la búsqueda en el estado global
 - `actions.findProductsCompleted`
 - La pantalla del resultado de la búsqueda se vuelve a renderizar mostrando el resultado de la búsqueda



Componente Products

```
import * as selectors from '../selectors';
import {ProductLink} from '../../common';

const Products = ({products, categories}) => (
```

```
  <Table striped hover>
```

```
    <thead>
```

```
      <tr>
```

```
        <th scope="col">
```

```
          <FormattedMessage id='...category' />
```

```
        </th>
```

```
        <th scope="col">
```

```
          <FormattedMessage id='project.global.fields.name' />
```

```
        </th>
```

```
      </tr>
```

```
    </thead>
```

React Bootstrap

- **Table** genera el tag `table` añadiendo estilos CSS

Componente Products

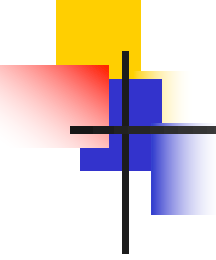
```
<tbody>
  {products.map (product =>
    <tr key={product.id}>
      <td>
        {selectors.getCategoryName(categories,
          product.categoryId) }
      </td>
      <td>
        <ProductLink id={product.id} name={product.name}/>
      </td>
    </tr>
  ) }
</tbody>
```

Se explica en el apartado 7.3

</Table>

);

export default Products;



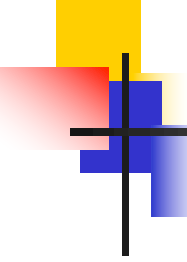
Acciones

```
import * as actionTypes from './actionTypes';

export const findAllCategoriesCompleted = ...

const findProductsCompleted = productSearch => ({
  type: actionTypes.FIND_PRODUCTS_COMPLETED,
  productSearch
});

const clearProductSearch = () => ({
  type: actionTypes.CLEAR_PRODUCT_SEARCH
});
```



Reducer

```
import {combineReducers} from 'redux';

import * as actionTypes from './actionTypes';

const initialState = {
  categories: null,
  productSearch: null
};

const categories = ...

const productSearch = (state = initialState.productSearch, action) => {
  switch (action.type) {
    case actionTypes.FIND_PRODUCTS_COMPLETED:
      return action.productSearch;
    case actionTypes.CLEAR_PRODUCT_SEARCH:
      return initialState.productSearch;
    default:
      return state;
  }
}
```



Reducer

```
const reducer = combineReducers({  
  categories,  
  productSearch  
});  
  
export default reducer;
```